SOLUTION to in-class practice and Homework

CMPT 295

Unit - Machine-Level Programming

Lecture 12 – Assembly language basics:
memory addressing modes,
`leaq` instruction and
arithmetic & logical operations

Practice and DEMO

For the 4th Practice Exercise on Lecture 11 Slide 18

if have 64-bit (8B) memory address space bytes

Details of `0x80(,%rdx,2)`

- Compute `0x80(,%rdx,2)` when `%rdx` contains `0xf000`
  One way to compute `0x80 + %rdx * 2` is as follows:

1. `0x80 + %rdx * 2 = 0x80 + 0xf000 * 2 = 0x80 + 0xf000 << 1`

`0xf000 << 1`

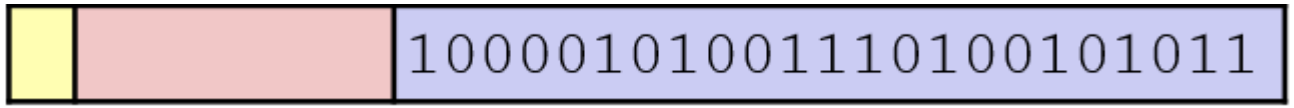
8B

1. `rdx` is 64-bit wide: `0xf000` = `0x000000000000f000`
   = $000000\ldots0000011110000000000000_2$
   48 0's
2. `0xf000 << 1` -> $000000\ldots0000011110000000000000_2$ << 1
   = $000000\ldots0000111100000000000000_2$
   47 0's
   = `0x000000000001e000` = `0x1e000`

2. `0x80 + 0x1e000 = 0x00080 + 0x1e000 = 0x1e080`

Review Questions

1. Rounding M = 1.10000101001110100101011101 would produce the following frac: True or False



2. These C statements print **1** (true) on the screen as the result of executing **aFloat == anInt**: True or False

```
float aFloat = 12345; int anInt = aFloat;
printf("%.2f == %d -> %d", aFloat, anInt, aFloat == anInt);
```

3. %rcx would contain **5** once the following instruction has executed True or False

```
movq 0x10(%rbx , %rax , 4), %rcx
```

if we have:

Registers	Value
%rax	0x0100
%rbx	0x0200
%rcx	128

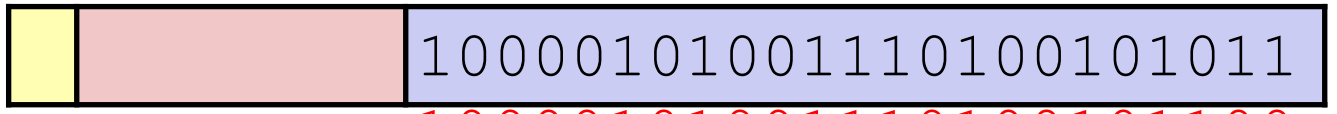
Address	Value
0x0600	5
0x0400	256
0x0610	8

Review Questions - SOLUTION

The discarded bits "101" ($1/2^{24} + 1/2^{26}$) > ½ of worth of bit in 23rd position ($1/2^{24}$), so we round up at 23rd bit and obtain:

Rounding position

1. Rounding M = 1.10000101001110100101011101 would produce the following frac:



10000101001110100101100

True or False

2. These C statements print **1** (true) on the screen as the result of executing **aFloat == anInt**:

True or False

```
float aFloat = 12345; int anInt = aFloat;
printf("%.2f == %d -> %d", aFloat, anInt, aFloat == anInt);
```

3. %rcx would contain **5** once the following instruction has executed

0x10 + 0x0200 + (0x0100 * 4) = 0x0610

movq 0x10(%rbx , %rax , 4), %rcx

movq (0x0610),%rcx

True or False

if we have:

Registers	Value
%rax	0x0100
%rbx	0x0200
%rcx	128

Address	Value
0x0600	5
0x0400	256
0x0610	8